Relatório de Auditoria de Segurança — Codehood Server —

20 de setembro de 2026

4	2	3	1
ALTA	MÉDIA	BAIXA	INFORMATIVA

Escopo	Repositório `codehood-server` completo: código-fonte, configuração, workflows de CI, scripts, build em `dist/` e o histórico do git (44 commits).
Revisão em	381f36c (branch main, arvore de trabalho suja)
Fora de escopo	CLI cliente (repositório separado), infraestrutura de hospedagem, dependências de terceiros (sem SCA).
Método	Leitura manual do código, com verificação pontual em runtime (Node 22) das hipóteses de explorabilidade.

Escopo e nota metodológica

A auditoria partiu da detecção da stack, porque as cinco categorias pedidas foram escritas para um projeto Supabase e precisam de tradução antes de fazerem sentido aqui. A tabela abaixo registra o que foi detectado; a seguinte, como cada categoria foi mapeada.

Camada	Detectado
Linguagem / runtime	TypeScript (ESM), Node >= 22.12
Framework	Astro 7 em modo `output: "server"`, adapter @astrojs/node standalone
Frontend	SolidJS + DaisyUI/Tailwind, ilhas dentro de páginas `.astro`
ORM / banco	Prisma 7 + SQLite (adapter better-sqlite3)
Autenticação	Cookie de sessão (token aleatorio, SHA-256 no banco) + Bearer API key
Autorização	Tabela declarativa de permissões em `src/auth/permissions/index.ts`
Superfícies de API	REST gerada por registry (`src/api/`), Astro Actions (`src/actions/`), JSON-RPC (`/rpc`)
Deploy	Nenhum Dockerfile/Helm/Terraform no repositório. CI: GitHub Actions (lint + testes)

Como cada categoria foi mapeada para esta stack

1. Banco sem tranca (isolamento)

Não há Supabase nem RLS. O isolamento é feito em duas camadas dentro das classes de serviço em `src/db/services/`: um fragmento `where` do Prisma (`courseWhere`, `userWhere`, `inviteWhere`, `questionWhere`, `courseContentsWhere`) que restringe a consulta, seguido de um laço `hasPerm(...)` sobre cada linha retornada. O inquilino é o curso (dono = instrutor; membros = matrículas ACTIVE). Auditamos toda consulta de listagem, busca e exportação a procura de uma das duas camadas ausente.

2. Permissão definida no navegador

Cada gate de papel do frontend (`hasPerm` em páginas `.astro`, `canManage`, `canWrite`, `adminMiddleware`, abas de `CourseHeader`) foi cruzado com o endpoint REST, a Astro Action e o método RPC correspondentes, verificando se o servidor repete a checagem.

3. IDOR

Percorremos todos os handlers das três superfícies: as rotas CRUD geradas por `src/api/registry/crud.ts` para 10 entidades, as 18 Astro Actions e os métodos RPC. Para cada uma verificamos se o objeto é carregado e conferido contra o ator antes de ser lido, alterado ou removido.

4. Chaves expostas

Varredura do código, de `.env`/`.env.example`, do workflow de CI, dos scripts e da documentação. Além de literais, procuramos defaults inseguros de variável de ambiente e ausência de validação de startup. O histórico completo do git (44 commits) foi varrido por segredos já commitados.

5. Inputs sem tratamento (XSS)

Sinks do Astro (`set:html`, `define:vars`) e do Solid (`innerHTML`), renderização de Markdown, `eval`/`new Function`, URLs controladas pelo usuário e montagem de cabeçalhos HTTP a partir da URL.

Convenção de severidade

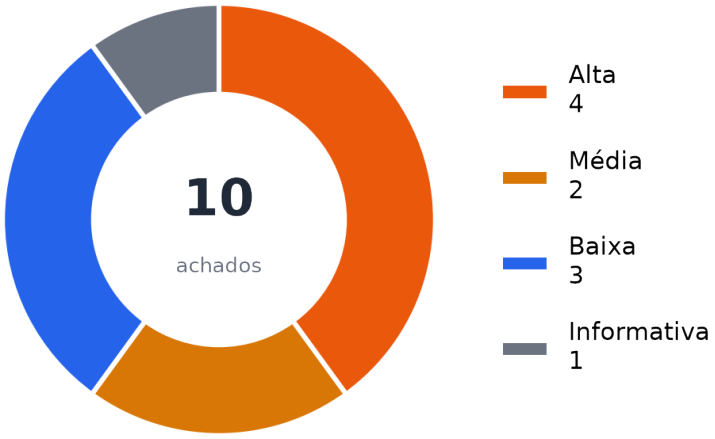
- **Critica** - comprometimento remoto sem autenticação, ou perda total de dados.
- **Alta** - escalada de privilégio, tomada de conta, ou leitura de dados de outro inquilino a partir de uma conta comum.

- **Media** - vazamento dé dado pessoal ou quebra de uma regra de negocio documentada, sem escalada direta.
- **Baixa** - endurecimento; hoje não explorável, mas a proteção depende de fator externo ao código.
- **Informativa** - decisão de projeto registrada, para revisão consciente no futuro.

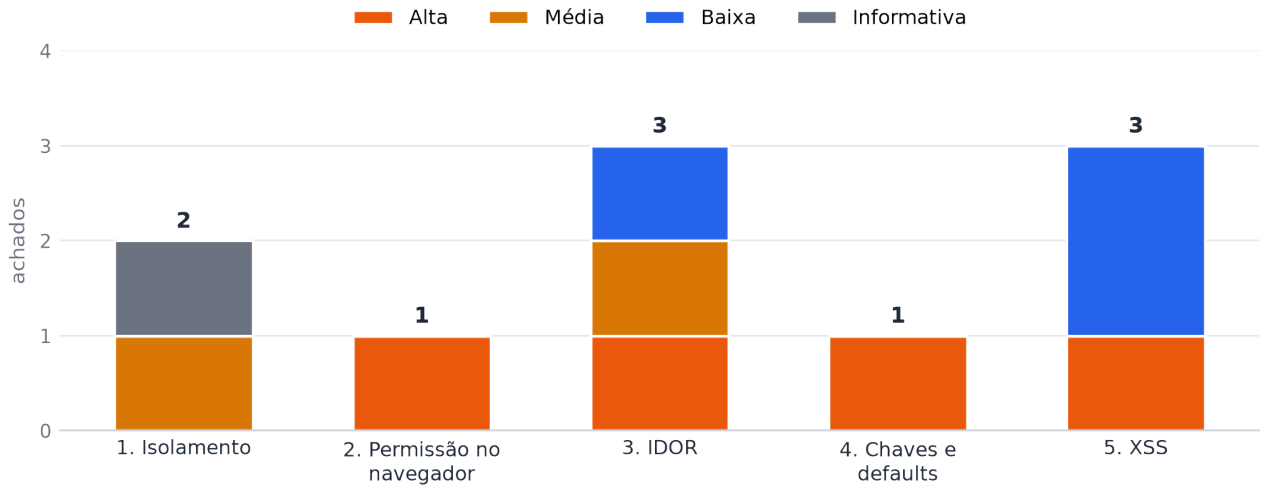
Resumo executivo

Foram levantados **10 achados** (9 acionaveis e 1 informativo), além de **12 pontos fortes** verificados. Nenhum achado critico: não há caminho de comprometimento sem autenticação, e a leitura de dados de outro curso sempre exige uma conta válida. Os quatro achados de severidade alta se concentram em dois temas. Três deles compartilham um padrão encadeável - o XSS do roster (F1) executa na sessão do instrutor, a troca de senha sem senha atual (F2) converte essa execução em tomada de conta, e o convite sem checagem de posse (F3) amplia o alcance para cursos alheios; corrigir F2 já quebra essa cadeia. O quarto (F4) é independente e vive na configuração de deploy: contas padrão criadas em runtime quando as variáveis de ambiente são esquecidas.

Achados por severidade



Achados por categoria



Índice dos achados

ID	Sev.	Categoria	Achado	Arquivo:linha
F1	ALTA	Inputs sem tratamento (XSS)	XSS armazenado no roster: JSON do aluno injetado dentro de <script> sem escape	`src/pages/[discipline]/[course]/roster.astro:448-452`

ID	Sev.	Categoria	Achado	Arquivo:linha
F2	ALTA	Permissão definida no navegador	Troca de senha pela API não exige a senha atual, ao contrario da UI	`src/api/index.ts:142-153`
F3	ALTA	IDOR	Convite de turma não valida a posse do curso: instrutor matrícula alunos no curso alheio	`src/db/services/invite.service.ts:72-97`
F4	ALTA	Chaves expostas e defaults inseguros	Contas padrão admin/admin criadas em runtime, atrás de dois interruptores de produção divergentes	`src/core/constants.ts:56-73`
F5	MÉDIA	Banco sem tranca (isolamento de dono/curso)	Lista completa de matriculados devolvida a qualquer aluno do curso	`src/core/schemas/course.ts:13`
F6	MÉDIA	IDOR	`createdBy` do convite vem do corpo da requisição, permitindo forjar a autoria	`src/core/schemas/invite.ts:21-28`
F7	BAIXA	Inputs sem tratamento (XSS)	`new MarkdownIt()` sem opções explicitas em schedule.astro, fora do padrão dos demais pontos	`src/pages/[discipline]/[course]/schedule.astro:23, 163`
F8	BAIXA	Inputs sem tratamento (XSS)	Nome de arquivo vindo da URL entra no Content-Disposition sem filtrar CR/LF	`src/utils/blob-response.ts:19-27`
F9	BAIXA	IDOR	`uploaderId` do anexo aceita outro usuário, desviando a contabilidade de quota	`src/db/services/attachment.service.ts:84-115`
F10	INFORMATIVA	Banco sem tranca (isolamento de dono/curso)	Blobs servidos sem autenticação, por decisão de projeto	`src/db/services/blob.service.ts:243-289`

Pontos fortes

Registrados com evidência, tanto porque são a prova de cobertura da auditoria quanto porque delimitam o que não precisa ser refeito ao corrigir os achados.

OK **Permissões declarativas com verificação em tempo de compilação**

`src/auth/permissions/index.ts:145-293`

Toda regra vive em um único objeto `PERMISSIONS`, e os tipos `_AssertPermissionsKeys`, `_AssertPermissionsUnique` e `_AssertPermissionsAudited` fazem o `tsc` recusar uma permissão com nome fora do vocabulario, duplicada ou sem redução de auditoria. Um `hasPerm` com alvo do tipo errado não compila.

OK **Isolamento em duas camadas nas listagens**

`course.service.ts:301-317, user.service.ts:310-313, invite.service.ts:278-282, question.service.ts:456-468`

Cada `findMany` combina um fragmento `where` do Prisma que restringe a consulta com um laço `hasPerm` sobre o resultado. Conferimos `course`, `user`, `invite`, `question`, `resource`, `exam`, `time-slot`, `calendar-event`, `api-key`, `passphrase` e `enrollment`: todas as onze aplicam pelo menos uma das camadas, e nove aplicam as duas.

OK **Rotas CRUD geradas, não escritas a mão**

`src/api/registry/crud.ts:307-527`

As 10 entidades REST compartilham um único gerador que delega ao serviço passando sempre `{ actor }`. Não há handler artesanal onde o ator possa ser esquecido, que é o modo de falha classico de IDOR em API própria.

OK **Saída podada pelo schema de resposta**

`src/api/registry/route.ts:211-215`

`options.out.parse(value)` remove qualquer campo não declarado antes do `Response.json`. E o que impede, por exemplo, `passwordHash` de escapar por `GET /api/user/[username]`, cujo `entity` e `userSchema.pick({ username, name })`.

OK **Gabarito de questão nunca sai da consulta**

`src/db/services/question.service.ts:65-87, 193-231`

`questionInclude(includePrivate)` decide na própria query se `privatePayload` é selecionado. O aluno não recebe o gabarito filtrado depois - ele nunca é lido do banco. A segunda consulta, com a metade privada, só é emitida após `course.update-contents` ser concedido.

OK **Guarda única para /admin**

`src/middleware.ts:42-50`

`adminMiddleware` exige `system.manage` para todo o prefixo `/admin`, em vez de duas linhas repetidas no topo de cada página. As Actions administrativas ainda assim revalidam no serviço (`edition.create`, `discipline.delete`, `user.create`, `session.manage`), de modo que a proteção não depende do caminho da URL.

OK **Credenciais armazenadas corretamente**

`src/auth/password.ts:10-18, src/auth/token.ts:6-14`

Argon2id com os parâmetros do baseline OWASP (m=19456, t=2, p=1). Tokens de sessão, API key e convite são 32 bytes de `randomBytes`, persistidos apenas como SHA-256 e exibidos em claro uma única vez. A força da senha e checada contra lista de comuns e contra vazamentos (`wildleek`).

OK **Nenhum segredo no código nem no histórico**

`.env.example, .github/workflows/ci.yml:78, 44 commits`

O único `.env` versionado é o exemplo, com apenas `DATABASE_URL`. O CI usa `${{ secrets.CODECOV_TOKEN }}`. A varredura dos 44 commits não encontrou chave, certificado ou banco commitado - apenas senhas de fixture de teste (`correct-horse-battery-staple`), que não são credenciais reais.

OK **Travessia de diretório barrada no armazenamento de blobs**

`src/db/services/blob.service.ts:298-315, 256-263`

`attachmentPath` resolve o caminho e recusa qualquer nome cujo diretório final saia do diretório do blob. `serve` valida o hash contra `blobHash` antes de tocar o disco, o que descarta `..` e maiusculas.

OK **Allowlist de MIME para exibição inline**

`src/utils/blob-response.ts:1-35`

Só imagem, audio, video e PDF são servidos `inline`; `image/svg+xml` e explicitamente excluido e todo o resto vira `attachment`, sempre com `X-Content-Type-Options: nosniff`. Um `.html` enviado por instrutor não executa na origem da aplicação.

OK **Autenticação resolvida antes do handler nas três superfícies**

`src/api/registry/route.ts:183-188, src/rpc/registry/index.ts:47-55, src/middleware.ts:42-50`

REST, RPC e páginas recusam o anônimo antes de validar parâmetros, o que evita usar mensagens de erro para sondar a forma de métodos que o chamador não pode invocar.

OK **Semântica de PUT consistente com a de POST**

`src/db/base-service.ts:134-145`

O helper `upsert` exige um `assertCreatable` que roda no ramo de update, garantindo que um `PUT` não conceda escrita mais ampla do que o `POST` equivalente - um caminho de escalada que costuma passar despercebido.

Pontos fracos

Os riscos centrais, lidos como padrões e não como defeitos isolados.

- O convite é a única escrita ligada a curso que não confere a posse do curso (F3), num codebase em que todas as outras conferem. E uma lacuna pontual, não sistêmica - mas concede matrícula, que é a chave de leitura de todo o conteúdo do curso.
- Duas regras de negocio existem apenas na Astro Action e não na rota REST equivalente: a senha atual na troca de senha (F2) e a autoria do convite (F6). O padrão do projeto - controlador fino sobre serviço - é o certo; esses dois casos escaparam dele.
- Configuração de produção depende de três interruptores diferentes (`ENVIRONMENT`, `NODE_ENV`, `import.meta.env.PROD`), todos com default inseguro e nenhum validado no startup (F4). O modo de falha é silencioso: o servidor sobe normalmente.
- `courseSchema` mistura a forma interna (matrículas usadas pela checagem de permissão) com a forma pública (F5), e o `out.parse` - que é a defesa contra vazamento de campo - obedece ao schema, então não ajuda quando o próprio schema esta largo demais.
- Um único `set:html` recebendo dado de usuário (F1) anula boa parte do cuidado do restante da aplicação, porque o alvo é sempre uma sessão mais privilegiada que a do atacante.

Achados detalhados

Agrupados pelas cinco categorias da auditoria e ordenados por severidade dentro de cada uma. Toda linha citada foi conferida no código; nenhuma hipótese sem verificação entrou aqui.

1. Banco sem tranca (isolamento de dono/curso)

MÉDIA **F5 - Lista completa de matriculados devolvida a qualquer aluno do curso**

`src/core/schemas/course.ts:13, src/db/services/course.service.ts:385-396, src/api/index.ts:25-38`

```
// src/core/schemas/course.ts:7-24
export const courseSchema = z.object({
  id: courseId,
  ...
  instructor: userInfo,
  enrollments: userInfo.array(),   // { name, username } de todos os matriculados
  ...
});
```

src/db/services/course.service.ts:385-396 - `fromDb` não filtra por ator

```
function fromDb(row: DbCourse, actor: Actor): Course {
  return {
    ...row,
    instructor: row.instructor,
    enrollments: row.enrollments.map((e) => ({
      username: e.username,
      name: e.user.name,
    })),
    joinedAt: joinedAtFor(row, actor),
  };
}
```

src/db/services/enrollment.service.ts:36-42 - a regra que o vazamento contradiz

```
/**
 * Dropping marks the row `DROPPED` rather than deleting it, so re-enrolling
 * keeps history. Students cannot list their classmates, a privacy default
 * rather than a technical limit.
 */
```

Por que é explorável. `enrollments` é carregado em `courseInclude` para alimentar a própria permissão `course.read` (decidir se o ator é membro) e o contador exibido no card, mas segue inteiro para a saída. Como `enrollments` esta declarado em `courseSchema`, o `options.out.parse(value)` de `src/api/registry/route.ts:215` - que existe justamente para podar campos não declarados - o preserva. Resultado: o aluno matriculado recebe nome e username de toda a turma em `GET /api/course/[discipline]/[course]`, em `GET /api/course` e no payload da página do curso. A permissão `enrollment.read`, que restringe a listagem ao dono do curso, é simplesmente contornada por outra rota.

Impacto. Divulgação do roster (nome civil + username) de toda a turma a qualquer colega, ao contrario da regra que o próprio serviço de matrícula documenta e aplica. Em um LMS isso é dado pessoal de terceiros; combinado com o F3, um invasor que se auto-matrícula em curso alheio extrai a turma inteira.

Condição de explorabilidade. Sem pré-condição: qualquer aluno com matrícula ACTIVE no curso.

Correção sugerida. Separar a forma interna da pública: manter `enrollments` no include do Prisma para a checagem de permissão, mas tirar do `courseSchema`, substituindo por `enrollmentCount` para o card. Quem precisa da lista já tem `enrollmentService.findMany`, que aplica `enrollment.read`.

Critérios de aceite

- `GET /api/course/...` autenticado como aluno matriculado não devolve `enrollments`.
- A mesma chamada como instrutor do curso ou admin mantem o comportamento esperado pela spec.
- As páginas que mostram a contagem passam a usar o novo campo.

- Teste de integração afirma a ausência do campo para o papel de aluno.

INFORMATIVA **F10 - Blobs servidos sem autenticação, por decisão de projeto**

`src/db/services/blob.service.ts:243-289, src/pages/files/[hash].ts:6-7`

```
/**
 * ... No authentication check by design
 * (FR-NFR-030, amended): the URL is the content's own hash, and nothing
 * whose disclosure matters is meant to live in a resource (FR-NFR-032).
 */
async serve(hash: string | undefined, name: string | undefined): Promise<Response> {
```

Por que é explorável. Escolha documentada: a URL é o hash do conteúdo e funciona como capability. Não é um controle esquecido. Registrado para que a decisão seja revisitada conscientemente se algum dia um recurso passar a conter material restrito (prova não publicada, gabarito em PDF, dado de aluno).

Impacto. Quem obtiver o hash le os bytes, sem sessão. Aceito pela spec.

Condição de explorabilidade. Por design. Requer conhecer o hash (256 bits) do conteúdo.

Correção sugerida. Nenhuma ação enquanto a premissa FR-NFR-032 valer. Se mudar, exigir `course.read-contents` sobre o recurso que aponta para o blob.

Critérios de aceite

- A premissa esta registrada na spec e revisada quando anexos passarem a carregar material restrito.

2. Permissão definida no navegador

ALTA **F2 - Troca de senha pela API não exige a senha atual, ao contrario da UI**

`src/api/index.ts:142-153, src/core/schemas/user.ts:40-51, src/db/services/user.service.ts:161-192`

```
updateMe: PATCH("/api/user/me", {
  in: schema.userUpdate,                      // userUpdate inclui `password`
  out: schema.userSchema.omit({ passwordHash: true }),
  handler: async (args) => {
    return db.user.update({ username: args.actor.username }, args.body, {
      actor: args.actor,
    });
  },
}),
```

src/actions/profile.ts:68-91 - o caminho da UI, que exige a senha atual

```
changePassword: defineAction({
  input: z.object({
    currentPassword: z.string().min(1),
    newPassword: z.string().min(8),
  }),
  handler: async (input, context) => {
    const actor = requireUser(context);
    const user = await db.user.findOne({ username: actor.username }, { actor });
    if (!user || !(await verifyPassword(user.passwordHash, input.currentPassword)))
      throw new ActionError({ code: "UNAUTHORIZED", ... });
    await db.user.updatePassword(user, input.newPassword, { actor });
  },
}),
```

src/db/services/user.service.ts:174-188 - o serviço só checa força da senha

```
if (password) {
  const issues = await passwordStrengthIssues(password);
  InvalidData.ensureNoError({ password: issues });
}
return toUser(await tx.user.update({
  where: { username: target.username },
  data: { ...rest, ...(password !== undefined
    ? { passwordHash: await hashPassword(password) } : {}) },
}));
```

Por que é explorável. A regra "para trocar a senha, prove que sabe a atual" existe apenas no handler da Astro Action que a tela `/profile` usa. O mesmo efeito é alcançado por `PATCH /api/user/me` com corpo `{"password": "..."}`, que passa pela permissão `user.update` (o ator e o próprio alvo) e reescreve o hash sem nenhuma reprova de credencial. O endpoint aceita tanto o cookie de sessão quanto uma API key Bearer.

Impacto. Qualquer captura momentânea de sessão vira posse permanente da conta: XSS (F1), uma estação deixada aberta, ou uma API key de robô de correção vazada em log de CI - todas se convertem em troca de senha e bloqueio do dono legítimo. Também remove a última barreira contra escalada quando a vítima é admin.

Condição de explorabilidade. Sem pré-condição: qualquer credencial válida (cookie ou Bearer) do próprio usuário.

Correção sugerida. Remover `password` de `userUpdate` e expor a troca de senha por uma rota própria (`POST /api/user/me/password`) que exija `currentPassword`, reutilizando a lógica de `profile.changePassword`. Trocar a senha deve também invalidar as demais sessões e API keys do usuário.

Critérios de aceite

- `PATCH /api/user/me` com `password` no corpo responde 400/404 e não altera o hash.
- Existe rota dedicada que exige `currentPassword` e a valida com `verifyPassword`.
- Trocar a senha revoga as demais sessões do usuário.
- Teste de integração cobre os dois caminhos.

3. IDOR

ALTA **F3 - Convite de turma não valida a posse do curso: instrutor matrícula alunos no curso alheio**

`src/db/services/invite.service.ts:72-97, src/auth/permissions/index.ts:193-198, src/actions/auth.ts:163-183`

```
protected async createTx(tx, input: InviteCreate, opts) {
  ensurePerm(opts.actor, "invite.create", { invitedRole: input.invitedRole });
  const token = generateToken();
  const invite = await tx.invite.create({
    data: {
      ...,
      courseId: input.courseId,          // <- nunca confrontado com o ator
      createdById: input.createdBy.username,
    },
    include: inviteInclude,
  });
```

src/auth/permissions/index.ts:193-198 - a regra só olha o papel convidado

```
"invite.create": {
  admin: (_, target) => target.invitedRole !== "ADMIN",
  instructor: (_, target) => target.invitedRole === "STUDENT",
  student: false,
  audit: (target) => ({ invitedRole: target.invitedRole }),
} satisfies PermDef<InviteCreateTarget>,
```

src/actions/auth.ts:57-116 - o resgate matrícula como SYSTEM, sem nova checagem

```
if (invite.courseId) {
  await db.enrollment.create(
    { courseId: invite.courseId, username: createdUser.username },
    { ...FULL_ACCESS, tx },
  );
}
```

Por que é explorável. `InviteCreateTarget` carrega apenas `invitedRole`, então a permissão não tem como olhar `courseId`. Nem a Action `createClassroomInvite`, nem `createPersonalInvite`, nem `POST /api/invite` carregam o curso para conferir o dono. Todas as demais escritas ligadas a curso (`enrollment.create`, `exam`, `question`, `resource`, `time-slot`, `calendar-event`, `passphrase`) fazem essa checagem - convite é a única que não faz. O resgate em `actions/auth.ts` roda como `FULL_ACCESS`, portanto também não repara o buraco.

Impacto. O instrutor A emite um link de turma para o curso do instrutor B. Quem resgatar vira aluno ACTIVE de B, e com isso ganha `course.read-contents`: provas publicadas, recursos, calendario e a lista completa de matriculados (ver F5) do curso de B. A permite popular o curso alheio com contas que controla, o que corrompé o roster e qualquer nota futura.

Condição de explorabilidade. Requer conta com papel INSTRUCTOR (ou ADMIN). Aluno é barrado por `student: false`.

Correção sugerida. Estender o alvo de `invite.create` para `{ invitedRole, course }` e exigir `enrollment.manage` (ou `course.update-contents`) sobre o curso quando `courseId` for informado; carregar o curso em `createTx` antes do `ensurePerm`, como fazem `timeSlotService.create` e `calendarEventService.create`.

Critérios de aceite

- Instrutor A recebe 403 ao criar convite com `courseId` de um curso de B, pela Action e pela rota REST.
- Admin que não leciona o curso segue o comportamento decidido e documentado na spec.
- Convite sem `courseId` continua funcionando para o papel permitido.
- Teste de regressão cobre o caso cruzado.

MÉDIA F6 - `createdBy` do convite vem do corpo da requisição, permitindo forjar a autoria

`src/core/schemas/invite.ts:21-28, src/db/services/invite.service.ts:90`

```
// src/core/schemas/invite.ts:21-28
export const inviteCreate = inviteSchema
  .omit({ id: true, expiresAt: true, createdAt: true, redemptions: true })
  .extend({ expiresInMs: z.number().int().optional() });
//  -> mantem `createdBy: userInfo`, vindo do cliente

// src/db/services/invite.service.ts:79-93
const invite = await tx.invite.create({
  data: { ..., createdById: input.createdBy.username },
});
```

Por que é explorável. `POST /api/invite` aceita `createdBy` do cliente e o grava sem confronto com `opts.actor`. As Astro Actions preenchem o campo corretamente com o ator, mas a rota REST e a superfície que a CLI usa. A permissão `invite.create` não inspeciona esse campo.

Impacto. Um instrutor emite convites que aparecem, para o admin, como emitidos por outra pessoa - a trilha de auditoria de `/admin` e o próprio campo `createdBy` deixam de ser confiaveis. Efeito colateral: como `invite.read` do instrutor filtra por `createdBy.username`, o convite forjado fica invisivel para quem realmente o criou e revogavel apenas pelo admin.

Condição de explorabilidade. Requer conta INSTRUCTOR ou ADMIN, com acesso a rota REST.

Correção sugerida. Remover `createdBy` de `inviteCreate` e derivar `createdById` de `opts.actor` dentro do serviço (`SYSTEM` continua precisando informa-lo explicitamente).

Critérios de aceite

- `POST /api/invite` com `createdBy` de outro usuário é rejeitado ou ignorado.
- O convite gravado tem sempre `createdById` igual ao ator (exceto chamadas SYSTEM).
- Teste cobre a tentativa de falsificação.

BAIXA **F9 - `uploaderId` do anexo aceita outro usuário, desviando a contabilidade de quota**

`src/db/services/attachment.service.ts:84-115`

```
const username = opts.actor === SYSTEM ? null : opts.actor.username;
return this.fromDbWithSource(
  await tx.attachment.create({
    data: {
      ...,
      uploaderId: input.uploaderId ?? username,   // input vence o ator
    },
  }), tx);
```

Por que é explorável. `assertWithinQuota` cobra a quota do ator, mas a linha gravada atribui a autoria a `input.uploaderId` quando presente. A classe esta documentada como "Not routed" e não aparece em `src/api/index.ts`: é alcançada apenas por `resourceService`, que já confere `course.update-contents`. Por isso o impacto hoje é contábil, não de acesso.

Impacto. Bytes atribuidos a outro usuário no ledger de armazenamento; sem efeito de acesso enquanto o serviço não for roteado.

Condição de explorabilidade. Não explorável por HTTP hoje: serviço não roteado. Listado porque vira um IDOR no dia em que for exposto.

Correção sugerida. Aceitar `uploaderId` apenas quando `opts.actor === SYSTEM`; caso contrario derivar do ator.

Critérios de aceite

- `uploaderId` diferente do ator é ignorado (ou rejeitado) fora de chamadas SYSTEM.
- Se o serviço for roteado no futuro, existe teste cobrindo a regra.

---

4. Chaves expostas e defaults inseguros

ALTA **F4 - Contas padrão admin/admin criadas em runtime, atrás de dois interruptores de produção divergentes**

`src/core/constants.ts:56-73, src/middleware.ts:74-95, src/db/bootstrap.ts:17-56`

```
// src/core/constants.ts:57
export const ENVIRONMENT = assertIn(readEnv("ENVIRONMENT", "dev"), ["dev", "prod"]);
export const PRODUCTION = ENVIRONMENT === "prod";
export const SESSION_COOKIE_OPTIONS = { httpOnly: true, secure: PRODUCTION, ... };

// src/middleware.ts:92-95
export const onRequest = env.ENVIRONMENT === "dev"
  ? sequence(...DEVELOPMENT_MIDDLEWARES, ...COMMON_MIDDLEWARES)
  : sequence(...COMMON_MIDDLEWARES);
```

src/db/bootstrap.ts:17-56 - outro interruptor, outra variável

```
export function ensureDevAdmin(): Promise<void> {
  if (process.env.NODE_ENV === "production") { ...; return Promise.resolve(); }
  devAdminPromise ??= createDevAdminIfMissing();
  return devAdminPromise;
}
...
await demoUser({ email: DEV_ADMIN_EMAIL, username: "admin", name: "Admin",
                 role: "ADMIN", password: DEV_ADMIN_USERNAME });   // senha = "admin"
```

src/actions/auth.ts:12-17 - um terceiro interruptor, resolvido em build time

```
const SESSION_COOKIE_OPTS = {
  httpOnly: true,
  secure: import.meta.env.PROD,
  sameSite: "lax" as const,
  path: "/",
};
```

Por que é explorável. Não há segredo em texto claro no repositório - o problema são os defaults. `ENVIRONMENT` tem default `"dev"`, e nada no startup recusa esse valor. Com ele, `devBootstrapMiddleware` entra na cadeia e roda `ensureDevAdmin()` no primeiro request. Essa função consulta uma variável *diferente*, `NODE_ENV`, e, se o banco estiver vazio, cria `admin/admin`, `instructor/instructor` e `student/student` mais os cursos de demonstração. Verificado no build: `dist/server/virtual_astro_middleware.mjs:16` mantem `process.env.NODE_ENV === "production"` como teste de runtime, ou seja, subir `node dist/server/entry.mjs` sem exportar nenhuma das duas variáveis basta. Como efeito colateral, `ENVIRONMENT` != `"prod"` também deixa o cookie de sessão sem a flag `Secure`. O README e o AGENTS.md não mencionam nenhuma das variáveis.

Impacto. Instância publicada com as variáveis de ambiente esquecidas fica com um administrador de senha `admin` acessivel na tela de login - comprometimento total é trivial de descobrir. Na mesma condição, o cookie de sessão trafega sem `Secure`.

Condição de explorabilidade. Requer deploy com `ENVIRONMENT` e `NODE_ENV` ausentes (ou não iguais a `prod`/`production`) e banco ainda sem usuários. Não é explorável em uma instância já povoada, porque `createDevAdminIfMissing` desiste quando encontra qualquer usuário.

Correção sugerida. Unificar em um único interruptor (`ENVIRONMENT`), sem default: falhar o boot se a variável não estiver definida. Mover o seed de desenvolvimento para fora do middleware, deixando-o apenas no script de seed explicito. Se as contas de demo permanecerem, gerar senha aleatoria e imprimi-la uma única vez no log.

Critérios de aceite

- Subir o servidor sem `ENVIRONMENT` definido falha com mensagem clara em vez de assumir `dev`.
- Nenhum caminho de request cria usuário; o seed só roda por `pnpm run db:seed`.
- `secure`, `httpOnly` e `sameSite` do cookie derivam de um único ponto compartilhado por `src/api/auth.ts` e `src/actions/auth.ts`.
- README documenta as variáveis obrigatórias de produção.

5. Inputs sem tratamento (XSS)

ALTA **F1 - XSS armazenado no roster: JSON do aluno injetado dentro de <script> sem escape**

`src/pages/[discipline]/[course]/roster.astro:448-452`

```
<script
  type="application/json"
  id="roster-csv-data"
  set:html={JSON.stringify(csvRows)}
/>
```

src/pages/[discipline]/[course]/roster.astro:93-103

```
const csvRows = [
  ["Name", "Username", "Email", "GitHub", "School ID", "Enrolled"],
  ...students.map((s) => [
    s.name, s.username, s.email, s.githubId, s.schoolId,
    s.enrolledAt.toISOString().slice(0, 10),
  ]),
];
```

src/core/schemas/user.ts:40-51 (userUpdate, aplicado pelo próprio aluno)

```
export const userUpdate = userSchema
  .pick({ name: true, email: true, githubId: true, schoolId: true })
  .extend({ password: z.string().min(1).optional() })
  .partial()
  .strict();
```

Por que é explorável. `set:html` é o escape hatch do Astro: o valor é escrito cru no HTML. `JSON.stringify` protege aspas e barras invertidas, mas não escapa `</script`, e o parser HTML encerra o elemento nesse ponto mesmo com `type="application/json"`. Os campos `name`, `email`, `githubId` e `schoolId` são gravados pelo próprio aluno via `actions.profile.update` ou `PATCH /api/user/me`, e `name` só exige `min(1)`. Verificado: `JSON.stringify(["</script><img src=x onerror=alert(1)>"])` devolve a sequência intacta.

Impacto. Um aluno matriculado define o próprio nome como `</script><img src=x onerror=...>`; o script roda toda vez que o instrutor (ou um admin) abre a aba Students do curso. O cookie de sessão é `httpOnly`, mas o payload age em nome da vítima na mesma origem: criar convites, apagar conteúdo do curso e - encadeando com o F2 - trocar a senha do instrutor sem conhecer a atual, o que é tomada de conta completa. Se a vítima for admin, é comprometimento total da instância.

Condição de explorabilidade. Sem pré-condição: basta uma conta de aluno matriculada no curso.

Correção sugerida. Não usar `set:html` para dados. Serializar com um escape de `<`/`>`/`&` (por exemplo `JSON.stringify(x).replace(/</g, "\\u003c")`) ou, melhor, passar as linhas por `define:vars`, que o Astro já escapa, ou montar o CSV em um endpoint autenticado.

Critérios de aceite

- O roster renderiza literalmente um aluno chamado `</script><img src=x onerror=alert(1)>` sem executar nada.
- Nenhum `set:html` do projeto recebe dado originado de entrada de usuário.
- Teste de regressão em `test/` cria um aluno com nome contendo `</script>` e afirma que a página do roster não contem `<img`.

BAIXA F7 - `new MarkdownIt()` sem opções explicitas em schedule.astro, fora do padrão dos demais pontos

`src/pages/[discipline]/[course]/schedule.astro:23, 163`

```
// linha 23
const md = new MarkdownIt();
// linha 163
set:html={md.render(event.description)}
```

Os outros três pontos fixam as opções

```
src/components/question/Markdown.tsx:12
  new MarkdownIt({ html: false, linkify: true, typographer: true })
src/pages/[discipline]/[course]/exams/[slug].astro:36
  new MarkdownIt({ html: false, linkify: true, typographer: true })
src/pages/[discipline]/[course]/resources/[slug].astro:43
  new MarkdownIt({ html: false }).render(data.content)
```

Por que é explorável. Hoje não é explorável: o default do markdown-it já é `html: false`, e seu `validateLink` bloqueia `javascript:`, `vbscript:` e `data:`. O achado é de consistência: este é o único dos quatro pontos de renderização cujo comportamento seguro depende do default de uma dependência em vez de estar escrito no código. Um bump de major version que mude o default transforma silenciosamente `event.description` - escrito pelo instrutor e sincronizado pela CLI - em HTML executavel na sessão de cada aluno.

Impacto. Nenhum hoje. Regressão silenciosa para XSS armazenado em caso de mudanca de default.

Condição de explorabilidade. Não explorável na versão atual de markdown-it (15.x). Condicional a mudanca de default upstream.

Correção sugerida. Extrair uma única instância compartilhada de MarkdownIt com `{ html: false }` explicito e usar nos quatro pontos.

Critérios de aceite

- Nenhum `new MarkdownIt()` sem `html: false` explicito no projeto.
- As quatro chamadas usam o mesmo helper.

BAIXA F8 - Nome de arquivo vindo da URL entra no Content-Disposition sem filtrar CR/LF

`src/utils/blob-response.ts:19-27, src/db/services/blob.service.ts:282-287, src/pages/files/[hash]/[name].ts:6-7`

```
// src/utils/blob-response.ts:19-27
function escapeFilename(filename: string): string {
  return filename.replace(/[\\"]/g, "\\$&");   // escapa \ e ", mas não CR/LF
}
export function contentDisposition(mimeType: string, filename: string): string {
  const disposition = isInlineMimeType(mimeType) ? "inline" : "attachment";
  return `${disposition}; filename="${escapeFilename(filename)}"`;
}
```

src/db/services/blob.service.ts:282-287 - `name` e o segmento cru da URL

```
const filename = name ?? attachment.filename;
const headers = new Headers({
  "Content-Type": attachment.mimeType,
  "Content-Disposition": contentDisposition(attachment.mimeType, filename),
  ...blobSecurityHeaders(),
});
```

Por que é explorável. `/files/[hash]/[name]` passa o segmento decodificado da URL direto para o cabeçalho. `%0d%0a` sobrevive a decodificação e a `escapeFilename`. Verificamos o comportamento no runtime alvo (Node 22, undici): `new Headers({"Content-Disposition": 'attachment; filename="a\r\nX-Injected: 1"'})` lanca `TypeError: invalid header value`. Portanto não há response splitting - a exceção escapa do handler e a rota devolve 500.

Impacto. Erro 500 em vez de 404 para uma URL maliciosa daquele blob; nenhum cabeçalho injetado. O risco real é de defesa em profundidade: a proteção vem do undici, não do código, e qualquer troca de runtime ou de caminho de resposta a remove.

Condição de explorabilidade. Parcialmente: causa 500, não injeção. Requer conhecer um hash de blob valido (que é público por design).

Correção sugerida. Sanitizar `name` com o mesmo `sanitizeFilename` usado em `attachmentService.create`, ou rejeitar qualquer caractere fora de `[\w.\- ]` antes de montar o cabeçalho.

Critérios de aceite

- `GET /files/<hash>/a%0d%0aX:1` responde 404 ou 400, nunca 500.
- O nome usado no cabeçalho passa por `sanitizeFilename`.

Recomendações priorizadas

P1 é o que deve entrar na próxima leva de trabalho: são os três achados que, encadeados, levam de conta de aluno a conta de administrador. P2 fecha as brechas de isolamento entre cursos. P3 é endurecimento.

Prio.	Ação	Detalhe
P1	**Fechar o XSS armazenado do roster (F1)**	Trocar `set:html={JSON.stringify(csvRows)}` por uma serialização que escape `<`, ou por `define:vars`. É o único achado que da execução de código na sessão de outra pessoa, e encadeia com o F2 até tomada de conta.
P1	**Exigir a senha atual em toda troca de senha (F2)**	Tirar `password` de `userUpdate`, criar rota dedicada com `currentPassword`, e invalidar as demais sessões e API keys após a troca. Isso também rebaixa o impacto do F1 de tomada de conta para ação pontual.
P1	**Definir o ambiente de produção de forma explicita e única (F4)**	Um só interruptor, sem default, com falha no boot se ausente; seed de desenvolvimento fora do middleware de request. Enquanto isso não for feito, publicar sempre com `ENVIRONMENT=prod` e `NODE_ENV=production` e conferir que não existe usuário `admin`.
P2	**Validar a posse do curso na criação de convite (F3)**	Ampliar o alvo de `invite.create` para incluir o curso e exigir `enrollment.manage` quando `courseId` for informado, alinhando o convite ao que as outras onze escritas de curso já fazem.
P2	**Separar a forma interna da pública em `courseSchema` (F5)**	Manter `enrollments` no include do Prisma para a checagem de permissão, remover do schema de saída, expor `enrollmentCount` para os cards.
P2	**Derivar `createdBy`/`uploaderId` do ator (F6, F9)**	Nenhum campo de autoria deve chegar do corpo da requisição, exceto em chamadas SYSTEM. Os dois casos são a mesma correção aplicada em dois serviços.
P3	**Padronizar a renderização de Markdown e o nome de arquivo servido (F7, F8)**	Uma única instância compartilhada de MarkdownIt com `html: false` explicito; `sanitizeFilename` aplicado também ao segmento `[name]` da rota de blobs.
P3	**Revisitar a premissa dos blobs públicos (F10)**	Manter a decisão enquanto FR-NFR-032 valer; anotar na spec o gatilho de revisão (o dia em que um recurso puder conter prova não publicada ou dado de aluno).

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Cada bloco vai de `--- ISSUE n ---` a `--- FIM ISSUE n ---`. Achados relacionados de baixo impacto foram agrupados numa issue só (F6+F9 na issue 6, F7+F8 na issue 7) para não gerar spam.

```
--- ISSUE 1 ---
```

```
**Título:** [Segurança] XSS armazenado no roster via JSON.stringify dentro de <script set:html>
**Labels:** security, severity:alta, área:frontend

## Problema

`src/pages/[discipline]/[course]/roster.astro` embute os dados do CSV do roster
em um `<script type="application/json">` usando `set:html={JSON.stringify(csvRows)}`.
`set:html` escreve o valor cru no HTML, e `JSON.stringify` não escapa a sequência
`</script`. O parser HTML encerra o elemento nesse ponto mesmo quando o `type` não
e JavaScript, então qualquer marcação após ela é interpretada como HTML.

As células de `csvRows` vêm de `name`, `email`, `githubId` e `schoolId` dos alunos
matriculados - todos campos que o próprio aluno grava por
`actions.profile.update` ou `PATCH /api/user/me`. `userUpdate` valida `name`
apenas com `z.string().min(1)`.
### Por que é explorável

1. Aluno matriculado no curso define o próprio nome como
   `</script><img src=x onerror=...>`.
2. O instrutor (ou um admin) abre a aba Students do curso.
3. O payload executa na origem da aplicação, na sessão da vítima.
O cookie de sessão é `httpOnly`, então não há roubo direto de cookie. Mas o
script age como a vítima na mesma origem: emitir convites, apagar conteúdo do
curso e, encadeando com a issue da troca de senha sem senha atual, assumir a
conta por completo.

## Evidência

`src/pages/[discipline]/[course]/roster.astro:448-452`
```astro
<script
  type="application/json"
  id="roster-csv-data"
  set:html={JSON.stringify(csvRows)}
/>
```
`src/pages/[discipline]/[course]/roster.astro:93-103`

```ts
const csvRows = [
  ["Name", "Username", "Email", "GitHub", "School ID", "Enrolled"],
  ...students.map((s) => [
    s.name, s.username, s.email, s.githubId, s.schoolId,
    s.enrolledAt.toISOString().slice(0, 10),
  ]),
];
```

`src/core/schemas/user.ts:40-51` - o aluno controla `name` sem restrição de
conteúdo:
```ts
export const userUpdate = userSchema
  .pick({ name: true, email: true, githubId: true, schoolId: true })
  .extend({ password: z.string().min(1).optional() })
  .partial()
  .strict();
```
Verificado em Node 22:
`JSON.stringify(["</script><img src=x onerror=alert(1)>"])` devolve a sequência
intacta, sem escape.

Impacto

Execução de código arbitrário na sessão do instrutor ou do admin que visitar o
roster. Aluno -> instrutor é escalada de privilégio; aluno -> admin e
```

```
comprometimento da instância.

## Sugestão de correção

Não usar `set:html` para transportar dados. Qualquer uma das opções:

- Escapar na serialização:
  `JSON.stringify(csvRows).replace(/</g, "\u003c")`.
- Passar as linhas por `define:vars`, que o Astro já escapa.
- Montar o CSV em um endpoint autenticado e baixar por `fetch`.

Vale varrer os demais `set:html` do projeto no mesmo passe; os outros usos
(`Tabs.astro`, as páginas de recurso e de prova) recebem HTML gerado por
`markdown-it` com `html: false`, não entrada crua, mas convém confirmar.

## Critérios de aceite
- [ ] Um aluno chamado `</script><img src=x onerror=alert(1)>` aparece
      literalmente na página do roster, sem executar nada.
- [ ] Nenhum `set:html` do projeto recebe dado originado de entrada de usuário.
- [ ] Teste de regressão em `test/` cria esse aluno e afirma que o HTML do
      roster não contem `<img`.
- [ ] `pnpm run lint` passa.
```

```
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
```

```
**Título:** [Segurança] PATCH /api/user/me troca a senha sem exigir a senha atual
**Labels:** security, severity:alta, área:api, área:auth

## Problema

A regra "para trocar a senha, prove que sabe a atual" existe apenas no handler
da Astro Action usada pela tela `/profile`. A rota REST equivalente não a tem.
`userUpdate` inclui `password`, e `PATCH /api/user/me` aceita esse schema
inteiro. O serviço só valida a força da nova senha e reescreve o hash.

### Por que é explorável

Qualquer posse temporária de uma credencial válida vira posse permanente da
conta:
- XSS (ver a issue do roster) executa o PATCH na sessão da vítima.
- Estação deixada aberta.
- API key Bearer vazada em log de CI ou no disco de um robô de correção -
  `apiKeyMiddleware` a converte em um ator completo, com o papel do dono.

Nenhum desses exige conhecer a senha atual, e após a troca o dono legítimo fica
sem acesso.
## Evidência

`src/api/index.ts:142-153`

```ts
updateMe: PATCH("/api/user/me", {
  in: schema.userUpdate,                       // userUpdate inclui `password`
  out: schema.userSchema.omit({ passwordHash: true }),
  handler: async (args) => {
    return db.user.update({ username: args.actor.username }, args.body, {
      actor: args.actor,
    });
  },
}),
```
`src/db/services/user.service.ts:174-188` - nenhuma reprova de credencial:

```ts
if (password) {
  const issues = await passwordStrengthIssues(password);
  InvalidData.ensureNoError({ password: issues });
}
return toUser(await tx.user.update({
```
```

```
  where: { username: target.username },
  data: { ...rest, ...(password !== undefined
    ? { passwordHash: await hashPassword(password) } : {}) },
}));
```

`src/actions/profile.ts:68-91` - o caminho da UI, que faz a checagem certa:
```ts
if (!user || !(await verifyPassword(user.passwordHash, input.currentPassword)))
  throw new ActionError({ code: "UNAUTHORIZED", ... });
```

Reprodução, autenticado com cookie de sessão ou Bearer:

```
PATCH /api/user/me
Content-Type: application/json

{"password": "nova-senha-qualquer"}
```

Impacto
Tomada de conta a partir de qualquer captura momentânea de sessão. Quando a vítima é admin, comprometimento total da instância. Também remove a última barreira que limitaria o impacto de um XSS.

Sugestão de correção

1. Remover `password` de `userUpdate`.
2. Criar rota dedicada, por exemplo `POST /api/user/me/password`, com `{ currentPassword, newPassword }`, reutilizando a lógica de `profile.changePassword`.
3. Invalidar as demais sessões e API keys do usuário após a troca (`db.session.delete({ username })`), para que a troca sirva também como resposta a incidente.

Decidir explicitamente o caso do admin trocando a senha de outra pessoa: se for para continuar existindo, que seja por rota separada e auditada, não pelo mesmo `userUpdate`.

Critérios de aceite

- [ ] `PATCH /api/user/me` com `password` no corpo não altera o hash.
- [ ] Existe rota dedicada que exige `currentPassword` e o valida com `verifyPassword`.
- [ ] Senha trocada revoga as demais sessões do usuário.
- [ ] Teste de integração cobre: troca com senha atual correta, com senha atual errada, e a tentativa pelo `PATCH`.
- [ ] `pnpm run lint` passa.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Criação de convite não valida a posse do curso
Labels: security, severity:alta, área:auth, área:api

Problema

`invite.create` só consulta `invitedRole`. O `courseId` do convite nunca e confrontado com o ator, nem no serviço, nem nas Astro Actions, nem na rota REST. Todas as demais escritas ligadas a curso carregam o curso e checam `course.update-contents` ou `enrollment.manage` antes de gravar (`enrollment.create`, `exam`, `question`, `resource`, `time-slot`, `calendar-event`, `passphrase`). Convite é a única que não faz.

Por que é explorável

1. Instrutor A chama `actions.auth.createClassroomInvite` (ou `POST /api/invite`) com o `courseId` de um curso do instrutor B.
2. A permissão passa: A é INSTRUCTOR e o papel convidado é STUDENT.
3. Quem resgatar o link vira matrícula ACTIVE no curso de B - o resgate em `src/actions/auth.ts` roda como `FULL_ACCESS`, então também não repara nada.
4. A matrícula concede `course.read-contents`: provas publicadas, recursos, calendario e a lista completa de matriculados do curso de B.

Evidência

`src/auth/permissions/index.ts:193-198` - o alvo não carrega o curso:

```ts
"invite.create": {
  admin: (_, target) => target.invitedRole !== "ADMIN",
  instructor: (_, target) => target.invitedRole === "STUDENT",
  student: false,
  audit: (target) => ({ invitedRole: target.invitedRole }),
} satisfies PermDef<InviteCreateTarget>,
```

`src/db/services/invite.service.ts:72-97`

```ts
ensurePerm(opts.actor, "invite.create", { invitedRole: input.invitedRole });
const invite = await tx.invite.create({
  data: { ..., courseId: input.courseId },   // nunca confrontado com o ator
});
```

`src/actions/auth.ts:163-183` - a Action repassa o `courseId` como veio:

```ts
createClassroomInvite: defineAction({
  input: z.object({ courseId: z.number().int(), maxUses: ... }),
  handler: withActionErrors(async (input, context) => {
    const actor = requireUser(context);
    const { token } = await db.invite.create(
      { ..., courseId: input.courseId as schema.CourseId, ... },
      { actor },
    );
```

`src/actions/auth.ts:109-114` - o resgate matrícula sem nova checagem:

```ts
if (invite.courseId) {
  await db.enrollment.create(
    { courseId: invite.courseId, username: createdUser.username },
    { ...FULL_ACCESS, tx },
  );
}
```

Impacto

Instrutor injeta contas que controla no curso de outro instrutor, ganhando leitura de todo o conteúdo do curso alheio e corrompendo o roster (e qualquer nota futura calculada sobre ele). Exige conta INSTRUCTOR ou ADMIN; aluno e barrado por `student: false`.

Sugestão de correção

Ampliar o alvo da permissão para `{ invitedRole, course }` e exigir `enrollment.manage` sobre o curso quando `courseId` estiver presente. Carregar o curso em `createTx` antes do `ensurePerm`, no mesmo formato que `timeSlotService.create` e `calendarEventService.create` já usam:

```ts
const course = input.courseId
  ? await tx.course.findUnique({
      where: { id: input.courseId },
      select: { id: true, instructor: { select: { username: true } } },
    })
  : null;
```

Decidir e documentar o comportamento do admin que não leciona o curso - a spec já trata `enrollment.manage` como direito de operação do curso, sem ramo de admin.

Critérios de aceite

- [ ] Instrutor A recebe 403 ao criar convite com `courseId` de curso de B, pela Action e pela rota REST.
- [ ] Convite sem `courseId` continua funcionando para o papel permitido.
- [ ] O comportamento do admin não-dono esta decidido, implementado e registrado na spec.
- [ ] Teste de regressão cobre o caso cruzado.
- [ ] `pnpm run lint` passa.

```
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
```

```
**Título:** [Segurança] Contas padrão admin/admin criadas em runtime por default de ambiente inseguro
**Labels:** security, severity:alta, área:config, área:deploy

## Problema

Não há segredo em texto claro no repositório. O problema são os defaults de
configuração, espalhados por três interruptores de produção diferentes, nenhum
validado no startup:
| Interruptor | Onde | Default | Resolvido em |
|---|---|---|---|
| `ENVIRONMENT` | `src/core/constants.ts:57` | `"dev"` | runtime |
| `NODE_ENV` | `src/db/bootstrap.ts:18` | ausente | runtime |
| `import.meta.env.PROD` | `src/actions/auth.ts:14` | - | build time |

Com `ENVIRONMENT` ausente, `devBootstrapMiddleware` entra na cadeia e roda
`ensureDevAdmin()` no primeiro request. Essa função consulta uma variável
diferente (`NODE_ENV`) e, com o banco vazio, cria `admin/admin`,
`instructor/instructor` e `student/student`.

Como efeito colateral, `ENVIRONMENT != "prod"` deixa o cookie de sessão sem a
flag `Secure` (`SESSION_COOKIE_OPTIONS`), enquanto `src/actions/auth.ts` decide
o mesmo atributo por outro critério, resolvido no build.
Nem o README nem o AGENTS.md mencionam essas variáveis.

### Por que é explorável

Verificado no build de produção: `dist/server/virtual_astro_middleware.mjs:16`
mantem `process.env.NODE_ENV === "production"` como teste de runtime, ou seja,
não foi inlinado. Subir `node dist/server/entry.mjs` sem exportar nenhuma das
duas variáveis, sobre um banco ainda sem usuários, basta para criar o
administrador de senha `admin`, acessivel pela tela de login pública.

## Evidência

`src/core/constants.ts:57-73`

```ts
export const ENVIRONMENT = assertIn(readEnv("ENVIRONMENT", "dev"), ["dev", "prod"]);
export const PRODUCTION = ENVIRONMENT === "prod";
export const SESSION_COOKIE_OPTIONS = { httpOnly: true, secure: PRODUCTION, ... };
```

`src/middleware.ts:92-95`

```ts
export const onRequest = env.ENVIRONMENT === "dev"
  ? sequence(...DEVELOPMENT_MIDDLEWARES, ...COMMON_MIDDLEWARES)
  : sequence(...COMMON_MIDDLEWARES);
```

`src/db/bootstrap.ts:17-38`

```ts
export function ensureDevAdmin(): Promise<void> {
  if (process.env.NODE_ENV === "production") { ...; return Promise.resolve(); }
  devAdminPromise ??= createDevAdminIfMissing();
  return devAdminPromise;
}
...
await demoUser({ email: DEV_ADMIN_EMAIL, username: "admin", name: "Admin",
                 role: "ADMIN", password: DEV_ADMIN_USERNAME });   // senha = "admin"
```
Impacto

Instância publicada com as variáveis esquecidas fica com administrador de senha
trivial, descoberto em uma tentativa. Na mesma condição, o cookie de sessão
trafega sem `Secure`.

Condição de explorabilidade: deploy com `ENVIRONMENT` e `NODE_ENV` ausentes (ou
```

```
diferentes de `prod`/`production`) e banco ainda sem nenhum usuário.
`createDevAdminIfMissing` desiste assim que encontra qualquer usuário, então uma
instância já povoada não é afetada.

## Sugestão de correção

1. Um único interruptor (`ENVIRONMENT`), sem default: falhar o boot com mensagem
   clara se a variável não estiver definida ou estiver vazia. `readEnv` hoje só
   testa `undefined`, então `ENVIRONMENT=""` também deveria ser recusado.
2. Tirar o seed de desenvolvimento do middleware de request; deixa-lo apenas em
   `pnpm run db:seed`.
3. Se as contas de demonstração permanecerem, gerar senha aleatoria e imprimi-la
   uma única vez no log, em vez de usar o username como senha.
4. Derivar `secure`/`httpOnly`/`sameSite` de um único ponto compartilhado por
   `src/api/auth.ts` e `src/actions/auth.ts`.
5. Documentar as variáveis obrigatórias de produção no README.
Mitigação imediata, enquanto isso não é feito: publicar sempre com
`ENVIRONMENT=prod` e `NODE_ENV=production`, e conferir que não existe usuário
`admin` no banco.

## Critérios de aceite

- [ ] Subir o servidor sem `ENVIRONMENT` (ou com valor vazio) falha com mensagem
      clara, em vez de assumir `dev`.
- [ ] Nenhum caminho de request cria usuário.
- [ ] Atributos do cookie de sessão vem de um único ponto.
- [ ] README lista as variáveis obrigatórias de produção.
- [ ] `pnpm run lint` passa.
```

```
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
```

```
**Título:** [Segurança] Lista completa de matriculados devolvida a qualquer aluno do curso
**Labels:** security, severity:media, área:api, área:privacy

## Problema

`courseSchema` declara `enrollments: userInfo.array()`, então a lista de
matriculados (nome e username de todos) acompanha toda leitura de curso.
`enrollments` é carregado em `courseInclude` por dois motivos legitimos:
alimentar a própria permissão `course.read` (decidir se o ator é membro) e o
contador exibido no card. Mas, por estar declarado no schema de saída, o
`options.out.parse(value)` de `src/api/registry/route.ts:215` - que existe
justamente para podar campos não declarados - o preserva.

O serviço de matrícula documenta e aplica a regra oposta:
```
Students cannot list their classmates, a privacy default rather than a
technical limit.
```

`enrollmentService.findMany` exige `enrollment.read`, que só o dono do curso
tem. Essa restrição e contornada por outra rota.
### Por que é explorável

Aluno com matrícula ACTIVE chama `GET /api/course/[discipline]/[course]` (ou
`GET /api/course`, ou simplesmente carrega a página do curso) e recebe a turma
inteira. Sem pré-condição além da matrícula.

## Evidência
`src/core/schemas/course.ts:7-24`

```ts
export const courseSchema = z.object({
  id: courseId,
  ...
  instructor: userInfo,
  enrollments: userInfo.array(),   // { name, username } de todos os matriculados
  ...
});
```

`src/db/services/course.service.ts:385-396` - `fromDb` não filtra por ator:

```ts
function fromDb(row: DbCourse, actor: Actor): Course {
```
```

```
  return {
    ...row,
    enrollments: row.enrollments.map((e) => ({
      username: e.username,
      name: e.user.name,
    })),
    joinedAt: joinedAtFor(row, actor),
  };
}
```

`src/db/services/enrollment.service.ts:36-42` - a regra contradita.

Impacto

Divulgação do roster (nome civil + username) de toda a turma a qualquer colega, contra a regra que o próprio codebase documenta. Em um LMS isso é dado pessoal de terceiros. Combinado com a issue do convite sem checagem de posse, um invasor que se auto-matrícula em curso alheio extrai a turma inteira.

Sugestão de correção

Separar a forma interna da pública:
- Manter `enrollments` em `courseInclude` (o Prisma precisa dele para a checagem de permissão).
- Remover `enrollments` de `courseSchema`; expor `enrollmentCount: z.number()` para os cards.
- Quem precisa da lista continua usando `enrollmentService.findMany`, que aplica `enrollment.read`.

Ajustar as páginas que hoje leem `course.enrollments` para o novo campo.
Critérios de aceite

- [ ] `GET /api/course/...` autenticado como aluno matriculado não devolve `enrollments`.
- [ ] A mesma chamada como instrutor do curso ou admin mantem o comportamento previsto pela spec.
- [ ] As páginas que mostram contagem usam `enrollmentCount`.
- [ ] Teste de integração afirma a ausência do campo para o papel de aluno.
- [ ] `pnpm run lint` passa.

```
--- FIM ISSUE 5 ---
```

```
--- ISSUE 6 ---
```

Título: [Segurança] Campos de autoria (createdBy, uploaderId) aceitos do corpo da requisição
Labels: security, severity:media, área:api

Problema

Dois serviços gravam a autoria a partir da entrada do cliente em vez de derivar do ator autenticado. Agrupados numa issue só por serem o mesmo defeito e a mesma correção.
1. `createdBy` do convite

`inviteCreate` mantem `createdBy: userInfo` vindo do cliente, e `createTx` grava `createdById: input.createdBy.username` sem confronto com `opts.actor`. As Astro Actions preenchem o campo corretamente, mas `POST /api/invite` - a superfície que a CLI usa - não.
Efeito colateral curioso: como o ramo de instrutor de `invite.read | invite.update | invite.delete` filtra por `target.createdBy.username === actor.username`, o convite forjado fica invisivel para quem realmente o criou, e só o admin consegue revoga-lo.

2. `uploaderId` do anexo

`attachmentService.create` cobra a quota do ator (`assertWithinQuota`), mas grava `uploaderId: input.uploaderId ?? username`. A classe esta documentada como "Not routed" e não aparece em `src/api/index.ts` - é alcançada apenas por `resourceService`, que já confere `course.update-contents`. Por isso hoje o impacto é contábil, não de acesso. Entra aqui para que vire um IDOR no dia em que o serviço for exposto.

Evidência

```
`src/core/schemas/invite.ts:21-28`

```ts
export const inviteCreate = inviteSchema
  .omit({ id: true, expiresAt: true, createdAt: true, redemptions: true })
  .extend({ expiresInMs: z.number().int().optional() });
//  -> mantem `createdBy: userInfo`, vindo do cliente
```
`src/db/services/invite.service.ts:79-93`

```ts
const invite = await tx.invite.create({
  data: { ..., createdById: input.createdBy.username },
});
```
`src/db/services/attachment.service.ts:100-111`

```ts
const username = opts.actor === SYSTEM ? null : opts.actor.username;
return this.fromDbWithSource(
  await tx.attachment.create({
    data: { ..., uploaderId: input.uploaderId ?? username },
  }), tx);
```

Impacto

- Convite: trilha de auditoria de `/admin` e o campo `createdBy` deixam de ser
  confiaveis; o convite forjado escapa da visão do suposto autor. Exige conta
  INSTRUCTOR ou ADMIN com acesso a rota REST.
- Anexo: bytes atribuidos a outro usuário no ledger de armazenamento. Sem efeito
  de acesso enquanto o serviço não for roteado.

Sugestão de correção

Regra única: nenhum campo de autoria chega do corpo da requisição, exceto em
chamadas `SYSTEM`, que não tem ator de quem derivar.
- Remover `createdBy` de `inviteCreate`; derivar `createdById` de `opts.actor`
  dentro de `createTx`, exigindo o campo explicito apenas quando
  `opts.actor === SYSTEM`.
- Mesmo tratamento para `uploaderId` em `attachmentService.create`.

Critérios de aceite

- [ ] `POST /api/invite` com `createdBy` de outro usuário é rejeitado ou
      ignorado; o gravado é sempre o ator.
- [ ] `uploaderId` diferente do ator é ignorado fora de chamadas SYSTEM.
- [ ] Chamadas SYSTEM (seed, comandos `manage`) continuam funcionando.
- [ ] Testes cobrem as duas tentativas de falsificação.
- [ ] `pnpm run lint` passa.
```

```
--- FIM ISSUE 6 ---
```

```
--- ISSUE 7 ---
```

```
**Título:** [Segurança] Padronizar renderização de Markdown e sanitização do nome de arquivo servido
**Labels:** security, severity:baixa, área:frontend, área:hardening

## Problema

Dois pontos de endurecimento. Nenhum é explorável hoje; ambos dependem de um
comportamento externo para continuar seguros, o que é exatamente o tipo de
proteção que some numa atualização de dependência.
### 1. `new MarkdownIt()` sem opções explicitas

`src/pages/[discipline]/[course]/schedule.astro:23` instância o markdown-it sem
argumentos e renderiza `event.description` com `set:html` na linha 163. Os
outros três pontos de renderização do projeto fixam as opções:

```
src/components/question/Markdown.tsx:12
  new MarkdownIt({ html: false, linkify: true, typographer: true })
src/pages/[discipline]/[course]/exams/[slug].astro:36
  new MarkdownIt({ html: false, linkify: true, typographer: true })
src/pages/[discipline]/[course]/resources/[slug].astro:43
  new MarkdownIt({ html: false }).render(data.content)
```
```

```
O default do markdown-it 15 já é `html: false`, e seu `validateLink` bloqueia
`javascript:`, `vbscript:` e `data:` - então não há falha hoje. O risco é que um
bump de major que mude esse default transforme silenciosamente
`event.description` (escrito pelo instrutor, sincronizado pela CLI) em HTML
executavel na sessão de cada aluno, sem que nada no código mude.

### 2. Nome de arquivo da URL no Content-Disposition
`/files/[hash]/[name]` passa o segmento decodificado da URL direto para o
cabeçalho. `escapeFilename` escapa `\` e `"`, mas não CR/LF, e `%0d%0a`
sobrevive a decodificação do path.

Verificado no runtime alvo (Node 22, undici):
`new Headers({"Content-Disposition": 'attachment; filename="a\r\nX-Injected: 1"'})`
lanca `TypeError: invalid header value`. Ou seja, não há response splitting - a
exceção escapa do handler e a rota devolve 500 em vez de 404. A proteção vem do
undici, não do código.

## Evidência

`src/utils/blob-response.ts:19-27`

```ts
function escapeFilename(filename: string): string {
  return filename.replace(/[\\"]/g, "\\$&");   // escapa \ e ", mas não CR/LF
}
export function contentDisposition(mimeType: string, filename: string): string {
  const disposition = isInlineMimeType(mimeType) ? "inline" : "attachment";
  return `${disposition}; filename="${escapeFilename(filename)}"`;
}
```
`src/db/services/blob.service.ts:282-287`

```ts
const filename = name ?? attachment.filename;
const headers = new Headers({
  "Content-Type": attachment.mimeType,
  "Content-Disposition": contentDisposition(attachment.mimeType, filename),
  ...blobSecurityHeaders(),
});
```

Impacto

- Markdown: nenhum hoje; regressão silenciosa para XSS armazenado se o default
  do markdown-it mudar.
- Content-Disposition: 500 em vez de 404 para uma URL maliciosa. Nenhum
  cabeçalho injetado no runtime atual.

Sugestão de correção

- Extrair uma única instância compartilhada de MarkdownIt com `html: false`
  explicito (por exemplo em `src/utils/`) e usa-la nos quatro pontos.
- Aplicar `sanitizeFilename` (já existente, usado em
  `attachmentService.create`) também ao segmento `[name]` antes de montar o
  cabeçalho, ou recusar qualquer caractere fora de `[\w.\- ]`.

Critérios de aceite

- [ ] Nenhum `new MarkdownIt()` sem `html: false` explicito no projeto.
- [ ] As quatro chamadas usam o mesmo helper.
- [ ] `GET /files/<hash>/a%0d%0aX:1` responde 404 ou 400, nunca 500.
- [ ] O nome usado no cabeçalho passa por `sanitizeFilename`.
- [ ] `pnpm run lint` passa.
```

```
--- FIM ISSUE 7 ---
```